算法模块直接调用手册（二次开发用）

> 本文档面向**二次开发者**，不使用 HTTP API，而是直接在 Python 中调用函数获取结果。
> 所有方法签名、参数类型、返回值结构均已对照源码验证。

目录

- 快速开始
- 1. 握力评估
- 2. 起坐评估
- 3. 静态站立评估
- 4. 步态分析
- 返回值速查表

快速开始

环境准备

```
cd algorithms
pip install -r requirements.txt
```

导入路径

```
import sys
sys.path.insert(0, 'algorithms')  # 如果不在 algorithms 目录下运行
```

两层调用方式

每个评估有两层可选：

层	适合场景	输入格式
渲染数据层（`*_render_data.py`）	二次开发推荐	Python 数组/列表
算法层（`*.py`）	需要 CSV 文本或更底层控制	CSV 字符串

推荐使用渲染数据层，因为：

1. 输入是数组，不需要自己组装 CSV
2. 提供 `get_*()` 方法按需提取各区域数据
3. 部分方法直接返回 ECharts option，方便可视化

---

1. 握力评估

文件

文件	行数	角色
`glove_render_data.py`	402	渲染数据封装（推荐入口）
`get_glove_info_from_csv.py`	1201	核心算法

总入口

```
from glove_render_data import generate_grip_report

result = generate_grip_report(
    sensor_data,      # list[list] 或 np.ndarray, shape [N, 256]
                      #   每行是一帧的256个传感器ADC值 (0~255)
    hand_type,        # str: '左手' 或 '右手'
                      #   决定传感器索引到手指的映射方式
    times=None,       # list[float] 或 None, shape [N]
                      #   每帧的时间戳(秒), None则按0.01s间隔自动生成
    imu_data=None,    # list[list] 或 None, shape [N, 4]
                      #   IMU四元数 [w, x, y, z], None则不计算欧拉角和角速度
)
```

内部调用链:

```
generate_grip_report()
  → 将数组组装为 CSV 字符串
  → 调用 process_glove_data_from_content(csv_content, hand_type)
  → 返回完整的 result dict
```

get_*() 方法详解

get_overview(result)

```
from glove_render_data import get_overview

overview = get_overview(result)
```

返回:

```
{
    'handType': '左手',            # str, 手类型
    'totalFrames': 1747,           # int, 总帧数
    'timeRange': '0.000s ~ 17.460s',  # str, 时间范围
    'totalForce': 58.5,            # float, 峰值帧总力(N)
    'totalArea': 240,              # int, 峰值帧总接触面积
    'peakInfo': {                  # dict, 峰值信息
        'peak_force': 85.2,        #   float, 峰值力(N)
        'peak_time': 6.340,        #   float, 峰值时间(s)
    }
}
```

get_time_analysis(result)

```
from glove_render_data import get_time_analysis

time_info = get_time_analysis(result)
```

返回:

```
[
    {'label': '抓握开始时间',   'value': '5.040 s'},
    {'label': '峰值力时间',    'value': '6.340 s'},
    {'label': '到达峰值耗时',   'value': '1.300 s'},
    {'label': '峰值区间开始',   'value': '5.800 s'},
    {'label': '峰值区间结束',   'value': '8.200 s'},
    {'label': '峰值持续时间',   'value': '2.400 s'},
    {'label': '峰值力',        'value': '85.20 N'},
    {'label': '检测阈值',      'value': '30°/s'},
    {'label': '抖动次数',      'value': '2 次'},
    {'label': '平均角速度',    'value': '12.50°/s'},
    {'label': '最大角速度',    'value': '45.30°/s'},
]
```

> 注意：抖动相关条目（检测阈值、抖动次数、平均/最大角速度）仅在提供了 `imu_data` 时才有值。

get_finger_data(result)

```
from glove_render_data import get_finger_data

fingers = get_finger_data(result)
```

返回:

```
[
    {'name': '大拇指',  'key': 'thumb',         'force': 12.5, 'area': 48, 'adc': 320,
'points': '24/30'},
    {'name': '食指',    'key': 'index_finger',  'force': 15.3, 'area': 52, 'adc': 410,
'points': '26/30'},
    {'name': '中指',    'key': 'middle_finger', 'force': 13.8, 'area': 45, 'adc': 380,
'points': '22/30'},
    {'name': '无名指',  'key': 'ring_finger',   'force': 8.2,  'area': 38, 'adc': 260,
'points': '19/30'},
    {'name': '小拇指',  'key': 'little_finger', 'force': 4.1,  'area': 25, 'adc': 150,
'points': '12/30'},
    {'name': '手掌',    'key': 'palm',          'force': 4.6,  'area': 32, 'adc': 180,
'points': '16/30'},
]
# force: 力(N), area: 有效面积(感应点数), adc: ADC原始值, points: 激活/总感应点
```

get_force_time_series(result)

```python
from glove_render_data import get_force_time_series

series = get_force_time_series(result)
```

返回:

```
{
    'times': [0.0, 0.035, 0.070, ...],       # list[float], 时间轴(秒), ~500个采样点
    'forceTimeSeries': {
        'thumb':         [0.0, 0.1, 0.3, ...],  # list[float], 大拇指力(N)
        'index_finger':  [0.0, 0.2, 0.5, ...],
        'middle_finger': [0.0, 0.1, 0.4, ...],
        'ring_finger':   [0.0, 0.0, 0.2, ...],
        'little_finger': [0.0, 0.0, 0.1, ...],
        'palm':          [0.0, 0.0, 0.1, ...],
        'total':         [0.0, 0.4, 1.6, ...],  # 总力
    }
}
```

get_force_time_echarts_option(result)

```python
from glove_render_data import get_force_time_echarts_option

option = get_force_time_echarts_option(result)
# 返回可直接传入 ECharts 的 option 对象 (dict)
# 包含: title, legend, xAxis, yAxis, series, tooltip, grid
```

get_force_distribution(result)

```
from glove_render_data import get_force_distribution

dist = get_force_distribution(result)
```

返回:

```
[
    {'name': '大拇指',  'key': 'thumb',         'force': 12.5, 'ratio': 0.2137},
    {'name': '食指',    'key': 'index_finger',  'force': 15.3, 'ratio': 0.2615},
    {'name': '中指',    'key': 'middle_finger', 'force': 13.8, 'ratio': 0.2359},
    {'name': '无名指',  'key': 'ring_finger',   'force': 8.2,  'ratio': 0.1402},
    {'name': '小拇指',  'key': 'little_finger', 'force': 4.1,  'ratio': 0.0701},
    {'name': '手掌',    'key': 'palm',          'force': 4.6,  'ratio': 0.0786},
]
# ratio: 占比 (0~1), 所有 ratio 之和 = 1.0
```

get_euler_data(result)

```
from glove_render_data import get_euler_data

euler = get_euler_data(result)
```

返回:

```
{
    'times': [0.0, 0.035, 0.070, ...],  # list[float], 时间轴(秒)
    'roll':  [2.1, 2.3, 2.5, ...],      # list[float], 横滚角(°)
    'pitch': [-5.0, -4.8, -4.6, ...],   # list[float], 俯仰角(°)
    'yaw':   [10.2, 10.1, 10.3, ...],   # list[float], 偏航角(°)
}
# 如果未提供 imu_data, 则 roll/pitch/yaw 为空列表
```

get_euler_echarts_option(result)

```
from glove_render_data import get_euler_echarts_option

option = get_euler_echarts_option(result)
# 返回 ECharts option, 3条线 (Roll/Pitch/Yaw)
```

get_angular_velocity_data(result)

```
from glove_render_data import get_angular_velocity_data

angular = get_angular_velocity_data(result)
```

返回:

```
{
    'times': [0.0, 0.035, 0.070, ...],           # list[float], 时间轴(秒)
    'angularVelocity': [5.2, 6.1, 8.3, ...],     # list[float], 角速度(°/s)
}
# 如果未提供 imu_data, 则 angularVelocity 为空列表
# 抖动检测阈值: 30°/s
```

完整调用示例

```
import sys
import numpy as np
sys.path.insert(0, 'algorithms')

from glove_render_data import (
    generate_grip_report,
    get_overview, get_time_analysis, get_finger_data,
    get_force_time_series, get_force_time_echarts_option,
    get_force_distribution, get_euler_data,
    get_euler_echarts_option, get_angular_velocity_data,
)

# 准备数据 (示例: 1000帧, 256个传感器, 有IMU)
N = 1000
sensor_data = np.random.randint(0, 100, (N, 256)).tolist()
times = [i * 0.01 for i in range(N)]
imu_data = [[0.98, 0.01, 0.02, 0.15]] * N  # 静态四元数

# 调用
result = generate_grip_report(sensor_data, '右手', times=times, imu_data=imu_data)

# 提取各区域
print("概览:", get_overview(result))
print("时间分析:", get_time_analysis(result))
print("各手指:", get_finger_data(result))
print("力时序:", len(get_force_time_series(result)['times']), "个采样点")
print("力分布:", get_force_distribution(result))
print("欧拉角:", len(get_euler_data(result)['roll']), "帧")
print("角速度:", len(get_angular_velocity_data(result)['angularVelocity']), "帧")

# 获取 ECharts 配置 (可直接传给前端)
force_option = get_force_time_echarts_option(result)
euler_option = get_euler_echarts_option(result)
```

2. 起坐评估

文件

文件	行数	角色
`sit_stand_render_data.py`	341	渲染数据封装（推荐入口）
`generate_sit_stand_pdf_v3.py`	1410	核心算法
`generate_ss_dashboard_v3.py`	810	动态视频生成
`layout_config.py`	274	PDF 布局配置

总入口

```
from sit_stand_render_data import generate_sit_stand_report

result = generate_sit_stand_report(
    stand_data,        # list[list] 或 np.ndarray, shape [N, 4096]
                       #   脚垫压力数据, 每行4096个值(64×64矩阵展平)
    sit_data,          # list[list] 或 np.ndarray, shape [M, 1024]
                       #   坐垫压力数据, 每行1024个值(32×32矩阵展平)
    username="用户",   # str, 报告中显示的用户名
)
```

内部调用链:

```
generate_sit_stand_report()
  → 将 stand_data 和 sit_data 分别组装为 CSV 字符串
  → 调用 generate_report_from_content(stand_csv, sit_csv, output_dir, username)
  → 返回完整的 result dict
```

get_*() 方法详解

get_duration_stats(result)

```
from sit_stand_render_data import get_duration_stats

stats = get_duration_stats(result)
```

返回:

```
{
    'total_duration': 26.84,   # float, 总测试时长(秒)
    'num_cycles': 5,           # int, 检测到的坐站周期数
    'avg_duration': 5.37,      # float, 平均周期时长(秒)
    'stand_frames': 1126,      # int, 站立阶段总帧数
    'sit_frames': 1126,        # int, 坐姿阶段总帧数
    'stand_peaks': 6,          # int, 检测到的站立力峰值数
    'username': '用户',         # str, 用户名
}
```

get_stand_evolution_images(result)

```
from sit_stand_render_data import get_stand_evolution_images

images = get_stand_evolution_images(result)
```

返回:

```
[
    {'label': 0, 'sublabel': 0,  'image': '...'},  # 左脚, 时间点0
    {'label': 0, 'sublabel': 1,  'image': '...'},  # 左脚, 时间点1
    ...
    {'label': 0, 'sublabel': 10, 'image': '...'},  # 左脚, 时间点10
    {'label': 1, 'sublabel': 0,  'image': '...'},  # 右脚, 时间点0
    ...
    {'label': 1, 'sublabel': 10, 'image': '...'},  # 右脚, 时间点10
]
# 共 22 张图 (2行×11列)
# label=0 左脚, label=1 右脚
# sublabel=0~10 对应周期进度 0%~100%
# image: base64 编码的 PNG 热力图
```

get_sit_evolution_images(result)

```
from sit_stand_render_data import get_sit_evolution_images

images = get_sit_evolution_images(result)
```

返回:

```
[
    {'label': 0,  'image': '...'},
    {'label': 1,  'image': '...'},
    ...
    {'label': 10, 'image': '...'},
]
# 共 11 张图 (1行×11列)
```

get_stand_cop_images(result)

```
from sit_stand_render_data import get_stand_cop_images

cop = get_stand_cop_images(result)
```

返回:

```
{
    'left':  '...',   # 左脚COP轨迹图(base64 PNG)
    'right': '...',   # 右脚COP轨迹图(base64 PNG)
}
```

get_sit_cop_image(result)

```
from sit_stand_render_data import get_sit_cop_image

cop = get_sit_cop_image(result)
# 返回: '...' (str) 或 None
```

get_force_curve_data(result)

```
from sit_stand_render_data import get_force_curve_data

curves = get_force_curve_data(result)
```

返回:

```
{
    'stand_times':     [0.0, 0.024, 0.048, ...],     # list[float], 站立力曲线时间轴
    'stand_force':     [12500, 13200, 14100, ...],    # list[float], 站立总力
    'sit_times':       [0.0, 0.024, 0.048, ...],     # list[float], 坐姿力曲线时间轴
    'sit_force':       [8500, 8600, 8700, ...],       # list[float], 坐姿总力
    'stand_peaks_idx': [42, 210, 378, 546, 714, 882], # list[int], 峰值帧索引
}
# stand_peaks_idx 可用于在图上标记周期分界线
```

get_stand_force_echarts_option(result) / get_sit_force_echarts_option(result)

```
from sit_stand_render_data import get_stand_force_echarts_option,
get_sit_force_echarts_option

stand_option = get_stand_force_echarts_option(result)
sit_option = get_sit_force_echarts_option(result)
# 返回: ECharts option dict, 可直接 setOption()
```

生成动态视频（独立方法）

```
from generate_ss_dashboard_v3 import generate_video_from_content

generate_video_from_content(
    stand_csv_content,      # str, 脚垫 CSV 文本
    sit_csv_content,        # str, 坐垫 CSV 文本
    output_path,            # str, 输出路径, 如 "output.mp4"
    speed_factor=0.5,       # float, 播放速度倍率
)
# 注意: 此方法接收 CSV 字符串, 不是数组
# 需要 matplotlib + ffmpeg
```

完整调用示例

```
import sys
import numpy as np
sys.path.insert(0, 'algorithms')

from sit_stand_render_data import (
    generate_sit_stand_report,
    get_duration_stats, get_stand_evolution_images,
    get_sit_evolution_images, get_stand_cop_images,
    get_sit_cop_image, get_force_curve_data,
    get_stand_force_echarts_option, get_sit_force_echarts_option,
)

# 准备数据
stand_data = np.random.randint(0, 50, (1200, 4096)).tolist()  # 脚垫 [1200帧, 4096点]
sit_data = np.random.randint(0, 30, (1200, 1024)).tolist()    # 坐垫 [1200帧, 1024点]

result = generate_sit_stand_report(stand_data, sit_data, username="测试用户")

print("周期统计:", get_duration_stats(result))
print("站立热力图:", len(get_stand_evolution_images(result)), "张")
print("坐姿热力图:", len(get_sit_evolution_images(result)), "张")

cop = get_stand_cop_images(result)
print("COP轨迹图:", "有" if cop.get('left') else "无")

curves = get_force_curve_data(result)
print("力曲线数据点:", len(curves['stand_times']))
```

3. 静态站立评估

文件

文件	行数	角色
`one_step_render_data.py`	387	渲染数据封装（推荐入口）
`OneStep_report.py`	2481	核心算法

总入口

```
from one_step_render_data import generate_standing_report

result = generate_standing_report(
    data_array,              # list[list] 或 np.ndarray, shape [N, 4096]
                             #   足底压力数据，每行4096个值(64×64矩阵展平)
    fps=42,                  # float, 采样率(Hz), 默认42
    threshold_ratio=0.8,     # float, COP计算阈值比例, 默认0.8
)
```

内部调用链：

```
generate_standing_report()
  → preprocess_origin_data(data_array, ...)    # 旋转、镜像、去噪、分离左右脚
  → cal_cop_fromData(processed, fps, ...)       # COP分析、足弓分类、摇摆特征
  → 组装 result dict
```

get_*() 方法详解

get_arch_overview(result)

```
from one_step_render_data import get_arch_overview

arch = get_arch_overview(result)
```

返回:

```
{
    'left': {
        'archIndex': 0.2345,       # float, 足弓指数
        'archType': 'Normal',      # str, 足弓类型: 'Flat'(扁平) / 'Normal'(正常) / 'High'(高
弓)
        'clarkeAngle': 42.5,       # float, Clarke角(°)
        'clarkeType': '正常足',     # str, Clarke分类
        'staheliRatio': 0.65,      # float, Staheli比
        'length': 25.3,            # float, 足长(cm)
        'width': 9.8,              # float, 足宽(cm)
        'totalArea': 142.5,        # float, 总接触面积(cm²)
```

```
        'forefootArea': 58.2,      # float, 前足面积(cm²)
        'midfootArea': 32.1,       # float, 中足面积(cm²)
        'hindfootArea': 52.2,      # float, 后足面积(cm²)
    },
    'right': {
        # 同上结构
    }
}
```

get_pressure_distribution(result)

```
from one_step_render_data import get_pressure_distribution

pressure = get_pressure_distribution(result)
```

返回:

```
{
    'left': {
        'forefoot': 45.2,    # float, 前足压力占比(%)
        'midfoot': 18.6,     # float, 中足压力占比(%)
        'hindfoot': 36.2,    # float, 后足压力占比(%)
    },
    'right': {
        'forefoot': 42.8,
        'midfoot': 20.1,
        'hindfoot': 37.1,
    }
}
# 每只脚的三个值之和 = 100%
```

get_arch_zone_data(result)

```
from one_step_render_data import get_arch_zone_data

zones = get_arch_zone_data(result)
```

返回:

```
{
    'leftSectionCoords': [
        [[x, y], [x, y], ...],   # 前足区域坐标
        [[x, y], [x, y], ...],   # 中足区域坐标
        [[x, y], [x, y], ...],   # 后足区域坐标
    ],
    'rightSectionCoords': [
        # 同上结构
    ],
    'peakFrameFlat': [0, 0, 5, 12, ...],  # list[int], 峰值帧4096个值(64×64展平)
}
# 坐标用于 Canvas 自定义绘制足弓分区
```

get_cop_trajectory_data(result)

```
from one_step_render_data import get_cop_trajectory_data

cop = get_cop_trajectory_data(result)
```

返回:

```
{
    'distLeftToBoth': 2.35,     # float, 左脚COP到整体COP的距离
    'distRightToBoth': 2.18,    # float, 右脚COP到整体COP的距离
    'leftForward': 0.82,        # float, 左脚COP前移量
}
```

get_cop_time_series(result) — 14项COP平衡参数

```
from one_step_render_data import get_cop_time_series

cop_params = get_cop_time_series(result)
```

返回:

```
{
    'pathLength': 245.8,        # float, COP轨迹总长度(mm)
    'contactArea': 1820.5,      # float, COP活动面积(mm²)
    'majorAxis': 18.6,          # float, 最大摇摆幅度(椭圆长轴)(mm)
    'minorAxis': 12.3,          # float, 最小摇摆幅度(椭圆短轴)(mm)
    'lsRatio': 1.51,            # float, 摇摆幅度系数(长轴/短轴)
    'eccentricity': 0.75,       # float, 摇摆均匀性系数(离心率)
    'deltaY': 15.2,             # float, 左右(ML)摇摆幅度(mm)
    'deltaX': 22.8,             # float, 前后(AP)摇摆幅度(mm)
    'maxDisplacement': 12.5,    # float, 最大偏心距(mm)
    'minDisplacement': 0.8,     # float, 最小偏心距(mm)
    'avgVelocity': 8.2,         # float, 平均COP速度(mm/s)
    'rmsDisplacement': 5.6,     # float, COP RMS位移(mm)
    'stdY': 3.8,                # float, 左右方向标准差(mm)
    'stdX': 5.2,                # float, 前后方向标准差(mm)
```

```
}
```

get_cop_metrics(result)

```
from one_step_render_data import get_cop_metrics

metrics = get_cop_metrics(result)
```

返回:

```
{
    'left': {
        # 左脚15项COP指标 (与 cop_time_series 类似, 但仅限左脚数据)
    },
    'right': {
        # 右脚15项COP指标
    }
}
```

get_sway_features(result)

```
from one_step_render_data import get_sway_features

sway = get_sway_features(result)
```

返回:

```
{
    'left': {
        # ML/AP方向摇摆位移、振荡频率、样本熵等
    },
    'right': {
        # 同上
    }
}
```

get_bilateral_pressure_ratio(result)

```
from one_step_render_data import get_bilateral_pressure_ratio

ratio = get_bilateral_pressure_ratio(result)
```

返回:

```
{
    'leftRatio': 48.5,     # float, 左脚压力占比(%)
    'rightRatio': 51.5,    # float, 右脚压力占比(%)
}
# leftRatio + rightRatio = 100%
```

完整调用示例

```python
import sys
import numpy as np
sys.path.insert(0, 'algorithms')

from one_step_render_data import (
    generate_standing_report,
    get_arch_overview, get_pressure_distribution,
    get_arch_zone_data, get_cop_trajectory_data,
    get_cop_time_series, get_cop_metrics,
    get_sway_features, get_bilateral_pressure_ratio,
)

# 准备数据 (示例: 500帧, 4096个传感点)
data_array = np.random.randint(0, 50, (500, 4096)).tolist()

result = generate_standing_report(data_array, fps=42, threshold_ratio=0.8)

# 足弓分析
arch = get_arch_overview(result)
print(f"左足弓类型: {arch['left']['archType']}")
print(f"右足弓Clarke角: {arch['right']['clarkeAngle']}°")

# 压力分布
pressure = get_pressure_distribution(result)
print(f"左脚前足占比: {pressure['left']['forefoot']}%")

# COP 14项参数
cop = get_cop_time_series(result)
print(f"COP路径长度: {cop['pathLength']}mm")
print(f"平均COP速度: {cop['avgVelocity']}mm/s")

# 左右压力比
ratio = get_bilateral_pressure_ratio(result)
print(f"左右压力比: {ratio['leftRatio']}% : {ratio['rightRatio']}%")
```

---

4. 步态分析

文件

文件	行数	角色
`gait_render_data.py`	378	渲染数据封装（推荐入口）
`generate_gait_report.py`	3235	核心算法（最大文件）

总入口

```
from gait_render_data import generate_gait_report

result = generate_gait_report(
    board_data,     # list[list[str]], 4块板数据
                    #   board_data[0]~[3] 对应第1~4块板
                    #   每块: list[str], 每个元素是 "[v0,v1,...,v4095]" 格式字符串
    board_times,    # list[list[str]], 4块板时间戳
                    #   每块: list[str], 每个元素是 "2025/12/06 17:07:33:840" 格式
)
```

内部调用链:

```
generate_gait_report()
  → _arrays_to_gait_csvs(board_data, board_times)   # 组装4个CSV字符串
  → analyze_gait_from_content(csv_contents)          # 核心分析
  → 返回完整 result dict (含 base64 图片)
```

> **注意:** `board_data` 的每个元素是**字符串** `"[v0,v1,...,v4095]"`，不是数组。这是因为压力垫原始输出就是这种格式。

get_*() 方法详解

get_gait_params(result) — 步态参数总览

```
from gait_render_data import get_gait_params

params = get_gait_params(result)
```

返回:

```
{
    'leftStepTime': '0.52s',          # str, 左脚步时
    'rightStepTime': '0.50s',         # str, 右脚步时
    'crossStepTime': '1.02s',         # str, 交叉步时(左+右)
    'leftStepLength': '62.5cm',       # str, 左脚步长
    'rightStepLength': '63.1cm',      # str, 右脚步长
    'crossStepLength': '125.6cm',     # str, 交叉步长
    'stepWidth': '12.3cm',            # str, 步宽
    'walkingSpeed': '1.23m/s',        # str, 步速
    'leftFPA': '8.5°',                # str, 左脚足偏角(FPA)
    'rightFPA': '9.2°',              # str, 右脚足偏角
    'doubleContactTime': '0.12s',     # str, 双支撑时间
}
# 注意: 值都是带单位的字符串格式
```

get_fpa_per_step(result) — 每步足偏角

```
from gait_render_data import get_fpa_per_step

fpa = get_fpa_per_step(result)
```

返回:

```
{
    'left':  [8.2, 9.1, 7.5, ...],    # list[float], 每步左脚FPA(°)
    'right': [9.0, 8.8, 9.5, ...],    # list[float], 每步右脚FPA(°)
}
```

get_balance(result) — 平衡分析

```
from gait_render_data import get_balance

balance = get_balance(result)
```

返回:

```
{
    'left': {
        '整足平衡': {'峰值': 125.3, '均值': 68.2,  '标准差': 22.1},
        '前足平衡': {'峰值': 95.2,  '均值': 42.5,  '标准差': 18.3},
        '足跟平衡': {'峰值': 88.7,  '均值': 38.9,  '标准差': 15.6},
    },
    'right': {
        # 同上结构
    }
}
```

get_time_series(result) — 时序曲线

```
from gait_render_data import get_time_series

ts = get_time_series(result)
```

返回:

```
{
    'left': {
        'time':     [0.0, 0.024, ...],   # list[float], 时间轴(秒)
        'area':     [12.5, 13.0, ...],    # list[float], 接触面积
        'force':    [125.3, 130.1, ...],  # list[float], 总力
        'copSpeed': [8.2, 9.1, ...],      # list[float], COP速度
        'pressure': [45.2, 48.1, ...],    # list[float], 平均压力
    },
    'right': {
        # 同上结构
    }
}
```

get_partition_features(result) — 6分区特征

```
from gait_render_data import get_partition_features

features = get_partition_features(result)
```

返回:

```
{
    'left': [
        # S1 (大拇趾)
        {'压力峰值': 85.2, '冲量': 12.5, '负载率': 0.21, '峰值时间_百分比': 65.3, '接触时间_百分
比': 78.2},
        # S2 (2~5趾)
        {'压力峰值': 62.1, '冲量': 8.3, '负载率': 0.15, '峰值时间_百分比': 72.1, '接触时间_百分比':
68.5},
        # S3 (前足)
        {...},
        # S4 (中足)
        {...},
        # S5 (外侧跟)
        {...},
        # S6 (内侧跟)
        {...},
    ],  # 共 6 个分区
    'right': [
        # 同上, 6个分区
    ]
}
```

6分区说明:

```
        足趾
    ┌─────────┐
    | S1 | S2 |    S1 = 大拇趾区域
    ├────┤    |    S2 = 2~5趾区域
    |    └────┤
    |   S3    |    S3 = 前足
    ├─────────┤
    |   S4    |    S4 = 中足
    ├────┬────┤
    | S6 | S5 |    S5 = 外侧足跟
    └────┴────┘    S6 = 内侧足跟
        足跟
```

get_partition_curves(result) — 6分区压力曲线

```python
from gait_render_data import get_partition_curves

curves = get_partition_curves(result)
```

返回:

```
{
    'left': [
        {'data': [0.0, 2.1, 5.3, ...]},   # S1 压力时序
        {'data': [0.0, 1.8, 4.2, ...]},   # S2
        {'data': [...]},                    # S3
        {'data': [...]},                    # S4
        {'data': [...]},                    # S5
        {'data': [...]},                    # S6
    ],
    'right': [
        # 同上, 6条曲线
    ]
}
```

get_region_coords(result) — 6分区坐标

```python
from gait_render_data import get_region_coords

coords = get_region_coords(result)
```

返回:

```
{
    'left': {
        'S1': [[x, y], [x, y], ...],   # 大拇趾区域的像素坐标列表
        'S2': [[x, y], ...],
        'S3': [[x, y], ...],
        'S4': [[x, y], ...],
        'S5': [[x, y], ...],
        'S6': [[x, y], ...],
    },
    'right': {
        # 同上
    }
}
```

get_support_phases(result) — 支撑相分析

```
from gait_render_data import get_support_phases

support = get_support_phases(result)
```

返回:

```
{
    'left': {
        '支撑前期': {
            '时长ms': 120.5,              # float, 该阶段时长(毫秒)
            '平均COP速度(mm/s)': 45.2,    # float, 平均COP速度
            '最大面积cm2': 28.3,          # float, 最大接触面积(cm²)
            '最大负荷': 125.6,            # float, 最大负荷
        },
        '支撑初期': {...},
        '支撑中期': {...},
        '支撑末期': {...},
    },
    'right': {
        # 同上, 4个阶段
    }
}
```

get_cycle_phases(result) — 步态周期分析

```
from gait_render_data import get_cycle_phases

cycle = get_cycle_phases(result)
```

返回:

```
{
    'left': {
        '双脚加载期': {
```

```
            '时长ms': 85.3,
            '平均COP速度(mm/s)': 38.5,
            '最大面积cm2': 22.1,
            '最大负荷': 105.2,
        },
        '左脚单支撑期': {...},
        '双脚摇摆期': {...},
        '右脚单支撑期': {...},
    },
    'right': {
        # 同上, 4个阶段
    }
}
```

get_images(result) — 所有图片

```
from gait_render_data import get_images

imgs = get_images(result)
```

返回:

```
{
    'pressureEvolution':   '...',  # 动态压力演变(2×10网格)
    'gaitAverage':         '...',  # 步态平均热力图+COP
    'footprintHeatmap':    '...',  # 足迹热力图+FPA线
    'timeSeries':          '...',  # 时序曲线图(4组)
    'leftPressureRegions': '...',  # 左脚分区热力图
    'rightPressureRegions':'...',  # 右脚分区热力图
    'leftPartitionCurves': '...',  # 左脚分区曲线
    'rightPartitionCurves':'...',  # 右脚分区曲线
}
# 共 8 张 base64 PNG 图片
# 可直接用 <img src={base64}> 显示
# 或用 base64.b64decode() 保存为文件
```

图片便捷方法

```
from gait_render_data import (
    get_pressure_evolution_image,   # → str (单张base64)
    get_gait_average_image,         # → str
    get_footprint_heatmap_image,    # → str
    get_time_series_image,          # → str
    get_pressure_region_images,     # → {'left': str, 'right': str}
    get_partition_curve_images,     # → {'left': str, 'right': str}
)
```

完整调用示例

```
import sys
sys.path.insert(0, 'algorithms')

from gait_render_data import (
    generate_gait_report,
    get_gait_params, get_fpa_per_step, get_balance,
    get_time_series, get_partition_features,
    get_support_phases, get_cycle_phases, get_images,
)

# 准备数据 — 4块板，每块若干帧
# 每帧是 "[v0,v1,...,v4095]" 格式的字符串
board_data = [
    ["[0,0,5,12,...共4096个值]", "[0,0,6,13,...]", ...],  # 板1
    [...],  # 板2
    [...],  # 板3
    [...],  # 板4
]
board_times = [
    ["2025/12/06 17:07:33:840", "2025/12/06 17:07:33:864", ...],  # 板1
    [...],  # 板2
    [...],  # 板3
    [...],  # 板4
]

result = generate_gait_report(board_data, board_times)

# 步态参数
params = get_gait_params(result)
print(f"步速: {params.get('walkingSpeed')}")
print(f"步宽: {params.get('stepWidth')}")

# 每步FPA
fpa = get_fpa_per_step(result)
print(f"左脚FPA: {fpa['left']}")

# 6分区特征
features = get_partition_features(result)
print(f"左脚S1(大拇趾)压力峰值: {features['left'][0]['压力峰值']}")

# 支撑相
support = get_support_phases(result)
print(f"左脚支撑前期时长: {support['left']['支撑前期']['时长ms']}ms")

# 图片保存
import base64
imgs = get_images(result)
for name, b64 in imgs.items():
    if b64:
        img_data = base64.b64decode(b64.split(',')[1])
```

```
        with open(f'{name}.png', 'wb') as f:
            f.write(img_data)
        print(f"已保存: {name}.png")
```

返回值速查表

握力 — glove_render_data.py (9个方法)

方法	返回类型	核心内容
`get_overview(result)`	dict	handType, totalFrames, timeRange, totalForce, totalArea, peakInfo
`get_time_analysis(result)`	list[dict]	11项时间指标 (抓握开始、峰值、抖动等)
`get_finger_data(result)`	list[dict]	6个部位: name, key, force, area, adc, points
`get_force_time_series(result)`	dict	times + 7通道力时序 (5指+掌+总力)
`get_force_time_echarts_option(result)`	dict	ECharts option (直接用)
`get_force_distribution(result)`	list[dict]	6个部位: name, key, force, ratio
`get_euler_data(result)`	dict	times, roll, pitch, yaw (需IMU)
`get_euler_echarts_option(result)`	dict	ECharts option (需IMU)
`get_angular_velocity_data(result)`	dict	times, angularVelocity (需IMU)

起坐 — sit_stand_render_data.py (8个方法)

方法	返回类型	核心内容
`get_duration_stats(result)`	dict	total_duration, num_cycles, avg_duration
`get_stand_evolution_images(result)`	list[dict]	22张base64热力图 (2×11)
`get_sit_evolution_images(result)`	list[dict]	11张base64热力图
`get_stand_cop_images(result)`	dict	left/right COP轨迹图
`get_sit_cop_image(result)`	str/None	坐姿COP轨迹图
`get_force_curve_data(result)`	dict	站立/坐姿力曲线 + 峰值索引
`get_stand_force_echarts_option(result)`	dict	ECharts option
`get_sit_force_echarts_option(result)`	dict	ECharts option

站立 — one_step_render_data.py (8个方法)

方法	返回类型	核心内容
`get_arch_overview(result)`	dict	左右脚: archIndex, archType, clarkeAngle, 尺寸, 面积
`get_pressure_distribution(result)`	dict	左右脚: forefoot/midfoot/hindfoot 占比(%)
`get_arch_zone_data(result)`	dict	分区坐标 + peakFrameFlat(4096)
`get_cop_trajectory_data(result)`	dict	COP距离指标
`get_cop_time_series(result)`	dict	**14项COP平衡参数**
`get_cop_metrics(result)`	dict	左右脚分别的COP指标
`get_sway_features(result)`	dict	ML/AP摇摆、频率、样本熵
`get_bilateral_pressure_ratio(result)`	dict	leftRatio/rightRatio(%)

步态 — gait_render_data.py (12个方法)

方法	返回类型	核心内容
`get_gait_params(result)`	dict	11项步态参数 (步长/步速/步宽/FPA/双支撑)
`get_fpa_per_step(result)`	dict	每步FPA列表
`get_balance(result)`	dict	整足/前足/足跟平衡 (峰值/均值/标准差)
`get_time_series(result)`	dict	5通道时序 (面积/力/COP速度/压力)
`get_partition_features(result)`	dict	6分区: 压力峰值/冲量/负载率
`get_partition_curves(result)`	dict	6分区压力曲线
`get_region_coords(result)`	dict	S1-S6坐标
`get_support_phases(result)`	dict	4个支撑阶段指标
`get_cycle_phases(result)`	dict	4个步态周期阶段指标
`get_images(result)`	dict	**8张base64图片**
`get_pressure_region_images(result)`	dict	左右分区热力图
`get_partition_curve_images(result)`	dict	左右分区曲线图

底层算法层直接调用

如果需要更底层的控制（如自定义 PDF 输出路径），可以直接调用算法层：

握力 — 算法层

```
from get_glove_info_from_csv import process_glove_data_from_content

result = process_glove_data_from_content(
    csv_content,      # str, CSV 文本 (格式:
sensor_data_calibrated,relative_time,imu_data_calibrated)
    hand_type,        # str, '左手' 或 '右手'
    output_dir=None,  # str 或 None, PDF/图片输出目录
)
```

起坐 — 算法层

```
from generate_sit_stand_pdf_v3 import generate_report_from_content

result = generate_report_from_content(
    stand_csv_content,  # str, 脚垫 CSV 文本
    sit_csv_content,    # str, 坐垫 CSV 文本
    output_dir=None,    # str 或 None, 输出目录
    username="用户",    # str
)
```

站立 — 算法层

```
from OneStep_report import preprocess_origin_data, cal_cop_fromData

# 第1步: 预处理
processed = preprocess_origin_data(
    raw_data,                        # np.ndarray, shape [N, 4096]
    rotate_90_ccw=True,              # 逆时针旋转90°
    mirrored_horizon=True,           # 水平镜像
    mirrored_vertical=True,          # 垂直镜像
    apply_denoise=True,              # 去噪
    small_comp_min_size=3,           # 小连通域最小尺寸
    small_comp_connectivity=4,       # 连通性
    margin=0,                        # 边距
    multi_component_mode=True,       # 多连通域模式(左右脚分离)
    multi_component_top_n=3,         # 取前N个最大连通域
    multi_component_min_size=10,     # 连通域最小尺寸
)

# 第2步: COP 分析
results = cal_cop_fromData(
    processed,
    threshold_ratio=0.8,
    fps=42,
    show_plots=False,                # 不弹出图形窗口
    save_pdf_path='report.pdf',      # 保存 PDF
    save_images_dir='./images/',     # 保存图片
)
```

步态 — 算法层

```
from generate_gait_report import analyze_gait_from_content

result = analyze_gait_from_content(
    csv_contents,       # list[str], 4个CSV文件的文本内容
    working_dir=None,   # str 或 None, 工作目录(用于存放临时图片)
)
```